2025 14th International Conference on System Modeling & Advancement in Research Trends (SMART) | 979-8-3315-6496-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/SMART66937.2025.11389274

Doc2GraphQA: Document to Knowledge Graph Question Answering System

R. Rajadurai[1], V. Naveen[2], M. Sakthivel[3], R. Asha[4], E. Naveen Kumar[5] and P. Eshwar[6]
[1-6]*Department of Artificial Intelligence and Data Science,*
Sri Shanmugha College of Engineering and Technology, Salem, India
E-mail: [1]rajadhurai.rajendhiran21@gmail.com, [2]naveenvisvanathan22@gmail.com,
[3]sakthisalem@gmail.com, [4]ashatvmd@gmail.com, [5]naveenkr211292@gmail.com, [6]eshwarkutty1234@gmail.com

***Abstract*—Doc2GraphQA is fundamentally a single framework that converts the extremely complex, multi-format documents into knowledge graphs that are semantically more meaningful in that you can perform fancy Q&A, aggregation, and predictive reasoning on big LLMs. It consists of three key components: (1) a Document Processor which reads PDFs, converts them to Markdown, disaggregates them into token-aware block, and adds metadata by use of a sequential hierarchical system that causes the LLM to generate summaries, roles, and relations. (2) a Knowledge Graph Builder which extracts entities and relations out of messy or semi-structured data, labels them with metadata like role, confidence, and source, and embeds the nodes in a hybrid symbolic-neural manner, allowing them to be accessed in such a manner. (3) a Retriever and Multi-Agent QA Engine that combines Cypher graph queries with a vector search that allows multi-hop reasoning, summarizing, and predictive inferences by specialized LLM agents. It is made to work with fixed and open-schema documents and is allegedly efficient and clear to analyze documents and solve complex tasks in the form of question and answer.**

Keywords: Document-to-Graph, LLMs, Semantic Parsing, Graph QA, Hybrid Retrieval, Metadata Generation, Knowledge Graphs, Chunking, Multi-Hop Reasoning, Document Understanding.

I. Introduction

At this point, big data is ubiquitous and corporations are using gigantic collections of unstructured text files and half-structured data in decision-making. They work with papers, guides, legal agreements, and bills, as well as CVs and all types of information that exist in unstructured text, charts, lists, or graphic form. As a case in point, we have graphs and diagrams interspersed. The most important is the possibility to capture such data and transform it into a transparent one and comprehend the intricacies or, otherwise, it is not an easy task to develop sophisticated tools of business, medicine, law, technology, research, and information systems [1] [2].

Although NLP is improving, the existing techniques of document-analysis continue to have a difficult time dealing with the complexity of structure and meaning of complex data. The keyword searches and flat text reps, or the hard-and-fast rule-based pipelines that are not quite adaptive and generalizable are still used by a lot of systems. LLM models are capable of reading and generating text, however, this does not deal with the larger issue of reasoning on the scale of entire documents. These models reach token limits, lose their context and occasionally give fabricated information in multi-step reasoning or inter-document work. They are also not so reliable when they have to connect semantically related, but spread-out information [3][4]. Moreover, the lack of transparency in the process of reaching decisions by the LLM contributes to a negative interpretability-related impact-particularly when it comes to high-stakes or regulated areas [5].

To seal this divide between the unstructured data in documents and the richer and more interpretive knowledge graphs, this paper presents Doc2GraphQA, a generalizable, extensible framework converting disordered document formats into more interpretable knowledge graphs. It helps in the advanced functionality such as Q&A, summarizing, and predictive reasoning. Doc2GraphQA is able to compute complex and heterogeneous scale docs in a transparent and reliable manner by integrating symbolic representations with neural language understanding [6][7].

In its essence, Doc2GraphQA possesses a high-level transformation pipeline that is intended to process smart content. It begins with a document processor which accepts PDFs (and any other format) and converts them to structured Markdown without altering the original layout and visual flow. The content is further divided into semantically independent, token-conscious parts. Every segment is subjected to a Sequential Hierarchical Metadata Generation System that generates contextual metadata: brief summaries, semantic roles such as background, methodology, results, cross-references with related parts, etc. through prompting with chain-of-thought. This metadata forms a primary semantic structure that retains the logical code and the story of the source [8][9].

Based on that, Doc2GraphQA builds a hybrid knowledge graph by combining factual relationships and semantic abstractions based on unstructured and semi-

structured data. In the case of raw text, entity recognition and relation extraction are promoted through LLM-directed prompting to write factual triples. The system uses layout-sensitive schema mapping to read relationships in semi structured stuff such as tables and lists. The metadata (provenance, confidence, semantic role, and contextual structure) is assigned to every node and edge. These graph fragments are represented in a dense vector space so that to enable both symbolic querying and neural similarity search [10][11].

In order to provide advanced access to information, Doc2GraphQA incorporates Retriever and Multi-Agent Reasoning Engine. This applies a hybrid retrieval, a combination of Cypher symbolic-queries and dense embedding similarity. Once the subgraph in question is extracted, a team of specialized LLM agents is invited to address complicated reasoning. They reason multi-hop graphically, combine information in remote document fragments, construct context summaries, and create insights of prediction such as selecting the most optimal option or predicting and conclusive results. The multi-agent system also allows each agent to specialize in a particular reasoning ability, resulting in the overall response to be correct and understandable [12][13][14]. We also have a verification agent that can check the output confidence twice and reduce the number of errors or fabricated results.

The Retriever and Multi-Agent Question Answering (QA) Engine is in the middle of the framework. A user query is initially converted to a Cypher query that specifies subgraphs that respond to the symbolic query to the user. Embedding-based similarity searches are then used to refine the subgraphs so as to allow it to identify semantically related but structurally concealed links. The highly refined subgraph is then processed by a group of LLM agents capable of doing multi-hop reasoning, merging facts across documents and segments, summarizing context, and providing predictive information such as selecting the most appropriate model to work with a dataset. To achieve more reliability, a verification agent is capable of assessing the trustworthiness of the output and identify any discrepancy.

The huge strength of Doc2GraphQA is that it is a highly flexible tool when it comes to all types of documents. Can deal with rigid formats such as structured resumes and financial statements it also works well with open schema stuff like research papers and legal documents. The system automatically adjusts its metadata schema and extraction mechanism to the appearance of whatever the document represents, thus avoiding the need to have previously created templates or manually tagging the data. The entire pipeline is constructed with the interpretability and auditability in mind; each step outputs its intermediate results and arguments, which means seeing how things are going wrong and preventing the error early.

Doc2GraphQA is by and large a significant advancement to document comprehension by converting static, multi-format documents into queryable and rich knowledge graphs. The system combines the ability to reason symbolically and process languages neurally into one, which remains scalable, interpretable, and intelligent for content analysis. It implies that you can apply it to expert Q&A, automated summarization, trend analysis, as well as multi-document synthesis, and therefore, it is bringing us to a new era of document intelligence that appreciates transparency and contextual awareness.

II. Related Work

The intersection of document understanding, semantic parsing and graph-based reasoning has been a hot topic of research recently. Scholars are taking unstructured text, converting it to structured forms, making big language model systems that take text as input, and combining symbolic logic and neural approaches to graph-structured data. The following discussion highlights some key basic contributions related to Doc2GraphQA, such as Cypher-based querying, hybrid retrieval methods, and knowledge-graph generation from documents.

Cypher Queries and Graph Reasoning: Cypher is a declarative graph query language which is a core tool of querying and managing property graphs in applications such as Neo4j. A number of works have examined combining LLMs with Cypher for QA based on knowledge graphs. GraphRAG presents a retrieval augmented generation system that utilizes Cypher to extract subgraphs for a language model. Equally, LLM-KGQA relies on templates to convert natural queries into Cypher queries enabling one to reason multi-hop between related entities. The translation of natural language to SPARQL has been done in earlier systems such as Neural SPARQL Machines, however Cypher with its flexible syntax has proven superior to domain-specific and semi-structured data. Doc2GraphQA is a new paradigm that automatically generates Cypher queries from chunk-level graph embeddings, which allows both symbolic and neural multi-agent reasoning.

Retriever-Based Systems Retriever-based QA systems combine the retrieval functionality of an LLM with communication capabilities to address context window issues. RAG introduced a two stage pipeline in that documents are indexed by using vector similarity, and retrieved chunks are handed into an LLM. Later, the retrieval sensitivity and multi-source summarization were improved with REALM and Fusion-in-decoder (FiD). Hybrid retrievers that incorporate both symbolic (e.g. keyword or graph-based) and neural (e.g. dense embedding) methods have appeared to work well in long document QA and reasoning. Modeling LLMs in a subgraph context with both Cypher and vector-based retrieval, KG-FiD and Graph - RAG++ improved performance. The Doc2GraphQA is an

extension of this to match graph embeddings to document metadata, and to use Cypher path traversal with a similarity search using vectors.

Knowledge Graph Construction from Documents Converting documents to knowledge graphs usually includes the process of entity and relation extraction, possibly enhanced with metadata. The classical pipelines were based on the rule-based NLP or supervised learning. More recent ways include schema-free extraction using LLMs, such as REBEL and SPoT, which use prompt-based generation of subject- predicate- object triples. In document-level settings, systems such as DocKG and Text2Graph put more emphasis on hierarchical segmentation of documents as well as layout-aware modeling. Doc2GraphQA features a Sequential Hierarchical Metadata Generation System, which incrementally adds information to each node including semantic roles, summarized information, and hierarchical context one at a time via the application of chain-of-thought prompting. This operation is semantically accurate and consistent and facilitates the ability to answer questions in a sophisticated manner on structured and unstructured document format.

III. Methodology

A. Overall Framework

Doc2GraphQA is fundamentally a framework that transforms complex documents into knowledge graphs that are simple to query and supports that with an interpretable, multi-agent, and Q&A interface. It takes in input such as a PDF, report or a table and forward it into the DocParser Engine. Such an engine is subdivided into three components namely the Extractor, the DocSegmenter, and the Sequential Metadata Generation System. Raw file is given to the Extractor, which spews out in MarkDown formatted in a structured format but preserves layout and structure. Then, compelling to the DocSegmenter chops the content into tokens aware semantically meaningful segments. Every chunk is enriched by chain-of-thought prompting metadata-chain-of-thought prompting is simply a fancy method of creating short summaries, providing semantic roles and connecting related pieces to each other.

These pieces of enriched texts and its metadata are entered into the Graph Constructor module. The metadata is embedded in the first stage, Pre-Retriever Engine and automatically forms Cypher queries to retrieve the relevant nodes. An entity, relationship, annotated nodes, semantic roles of nodes, provenance and confidence scores are then stored in a graph database. Cypher Query Generator on the basis of the LLM is also useful in creating symbolic queries on the basis of metadata or any input by the user.

By asking a question, a Retriever Engine (consisting of a Question Understanding component and Multi-Agent System) processes it. It performs hybrid retrieval, combining symbolic Cypher queries and semantic search, which is a vector-based search. Specialised LLM agents process the retrieved subgraphs and make multi-hop reasoning, data aggregation, contextual summarization, and predictive analyses. This entire flow keeps on the responses precise, interpretable, and flexible to a number of document types and tough queries.

B. DocParse Engine

The DocParser Engine converts varied input papers to structured and semantically rich pieces to the graph stage. It is divided into three sub modules namely: the Extractor, the DocSegmenter, and the Sequential Metadata Generation System (SMGS).

1) Extractor (with OCR, VLMs, and Markdown Output)

Therefore, the Extractor is what our Doc2GraphQA project essentially is and with the Extractor being the first step. It accepts any type of document we toss at it, whether it is an unfiltered PDF, Word document, scanned document, or even a weird web page, after which it vomits out a Markdown file serving all the structure and visual complexities in place. It supports a number of files:

- Born-digital printed materials (PDFs, word papers, cold-fashioned HTMLs, etc.)
- Scanned documents (e.g., images, faxes)
- Graphic designs involving graphs, charts or tables.

In fact, the Extractor employs two basic technologies.

A. OCR Engine

In the case of photos or scans, we use OCR using software such as Tesseract, PaddleOCR or commercial engines such as Google Cloud Vision or Azure OCR. OCR not only extrudes the text and the bounding boxes and write notes, but also to put everything in a natural reading sequence. In order to organize the text we then divide the text into layout parts:

- Headings, identified by differences in font size or emphasis (e.g., bold text)
- Stanzas, marked out by spacing and lines.
- Reading-Lists and tables which are distinguished by indent and aligning marks.

B. Vision-Language Model (VLM) Parsing

We switch to such places as Vision\-Language Introduction models as LayoutLMv3, Donut, and Pix2Struct when we need to address very complex visual layouts. These tools are basically a combination of text and visual cues simultaneously to ensure that we get the job done.

- Identify tables, figures, and captions
- Semantic tag of a corresponding semantic meaning (e.g. “Table 1”, “Abstract”, “Conclusion”)
- Extract data in diagrams, form fields and charts.

In addition, they provide OCR a very fine fine-tuning of mistranslated text using context-aware embeddings.

C. Markdown-Based Structured Output

The entire content that we extract is imported into a clean Markdown that displays the page structure and simplifies the division of the content at a later date. The rules we follow are:

- > Figure/Table: to label any chart or picture.
- # for top-level sections (e.g., Title, Abstract)
- ## in the case of subsections (as of Method or Results)
- **Summary:** for key sentences abstracted using LLM summarization
- - in small things or list of bullets.

1) DocSegmenter (Markdown-Aware, Token-Limited Chunking with Semantic Alignment)

The DocSegmenter is the second component of the DocParser Engine. It reads the detachment of the Markdown produced by the Extractor and splits it into pieces that consist of tokens and continue to be readable whole. It does not simply chop at the sentences or paragraph divisions, but considers the hierarchy of the document, how the visuals are organized and natural topic transitions so that the blocks do retain their context and mimic the way a human reader would normally transition between topics.

A. Markdown-Aware Sectioning

The DocSegmenter examines Markdown the Extractor renders and identifies things such as #, ##, and list bullets (-, 1.) to determine where one section starts and another one begins. It then collects any given section that it identifies, and refines it into a block that fits within the token boundaries that we are interested in.

Example:

2. Incident Categories
- Malware Attacks: 45%
- Phishing: 30%
> Figure 2: Pie chart of type of incident distribution

The system identifies that this block is a coherent unit due to the fact that it has got a clear heading and a well-formatted list.

B. Token-Aware Chunk Splitting

The Markdown can be divided after which another module intervenes to predict the number of tokens each portion of the Markdown would consume in a large language model such as GPT-4 or BERT. When a section reaches our target threshold (e.g. 512 or 1024 tokens), we divide it into parts with the help of a combination of the following rules:

- Sentence boundaries
- Paragraph endings
- List/item separators
- Table row boundaries

To maintain the meaning through the slices, the tool employs a sliding-window (or overlapping) strategy with a 20% overlap in that, therefore, each chunk has a small amount of context with its neighbor.

C. Semantic Transition Detection (Optional VLM-Aware)

On documents with dense or ambiguous structures the DocSegmenter uses semantic shift detection methods that use Vision-Language Models (VLMs) or lightweight encoders like SBERT and RoBERTa. The methodology allows identifying the topic shifts even in situations when the structural indicators are low or ambiguous. RoBERTa. This aids in the identification of topic changes in the case where structural markers are weak.

Example Techniques

- Cosine similarity decrease between consecutive sentences# for top level sections (e.g. Title, Abstract)
- Heading style adjustments (e.g. bold/italic, which is detected through VLM)
- Caption-to-paragraph divergence (e.g. as seen in Fig. 3) to new topic.

D. Chunk Metadata Assignment

The metadata to each generated chunk will be:

- chunk_id: Unique identifier
- section_title: Extracted from Markdown (e.g., “2. Incident Categories”)
- token_count: Estimated length
- start_pos / end_pos: Byte offset in original file (if traceability is needed)
- source_format: e.g., Markdown, OCR, VLM
- chunk_text: The actual content string

Chunk Output Example:

json

{
“chunk_id”: “chunk_12”, “section_title”:
“2. Incident Categories”, “token_count”: 132,
“chunk_text”: “Malware Attacks: 45% ...
Incident distribution is indicated in figure 2”,
“source_format”: “Markdown”, “page”: 3
}

E. Integration with Metadata Generation

The segments of this step, which are token aligned, are fed into the SMGS. It cleanses them up with brief summaries, labels every section with its purpose and connects it to other sections. This ensures that each chunk is self-contained, meaningfully contextual and aligned to visual and semantic boundaries, which is beneficial in terms of the ability to accurately answer multi-hop questions and build graphs in the future.

2) Sequential Metadata Generation System (SMGS)

The DocParser Engine final component is the SMGS. It constructs hierarchical, extensive metadata of each segment DocSegmenter produces. That metadata forms the basis of building the knowledge graph and that it is the basis of multi-agent question answering. Possibly unlike conventional rule-based tags, the SMGS models large language models, chain-of-thought prompting, and cross-segment memory to predict document-level roles, relationships and summaries whilst being semantically correct and structurally consistent.

A. Objective and Motivation

Papers, policy documents and tech manuals are typically divided into parts such as background, methods, results, conclusions, etc. in real life. This is not always well marked on those parts. In order to gain proper insight into a piece of writing, we have to track down every role, observe the way in which we can connect sections, and gain a deeper meaning. That is why the SMGS was developed to unveil that concealed form and present it in the clear, systematic form.

$M_i=(s_i,r_i,h_i,\rho_i,\gamma_i)$

Where:

- s_i: abstractive summary
- r_i: the role of the semantics (e.g., Background, Methodology)
- h_i: the level of hierarchy (e.g., Section> Subsection)
- ρ_i: cross-references to other chunks.
- γ_i: confidence score

B. Chain-of-Thought Metadata Inference

MGS uses a multi-step prompting process to direct the LLM to follow a interpretable reasoning in steps that are clear.Each segment the LLM is prompted to:

- To summarize the importance of the chunk
- Categorize the semantic role according to the discourse features (e.g. linguistic features, os, transitions, etc.).
- ConContextual infer the hierarchical context from surround headers and struct tokens.
- Work out implicit or explicit cross-references (e.g. as shown in Fig. 3) to chunks (e.g. chunk 15).
- Rating the confidence of its inference.

This design guarantees stability of multi-domain environments, both fixed- schema and open- schema.

Example Prompt Chain (abbreviated):

[Chunk]: We have seen that by tuning the model performance can be increased 12 percent...

Step 1: What is the key learning? 12% improvement. with tuning"

Step 2: Click on the type of section to be chosen = Results.

Step 3: It includes reference to other segments, yes, previous experiments.

This logical method of reasoning is stepwise and will reduce hallucinations and provide logical consistency in all parts of the document.

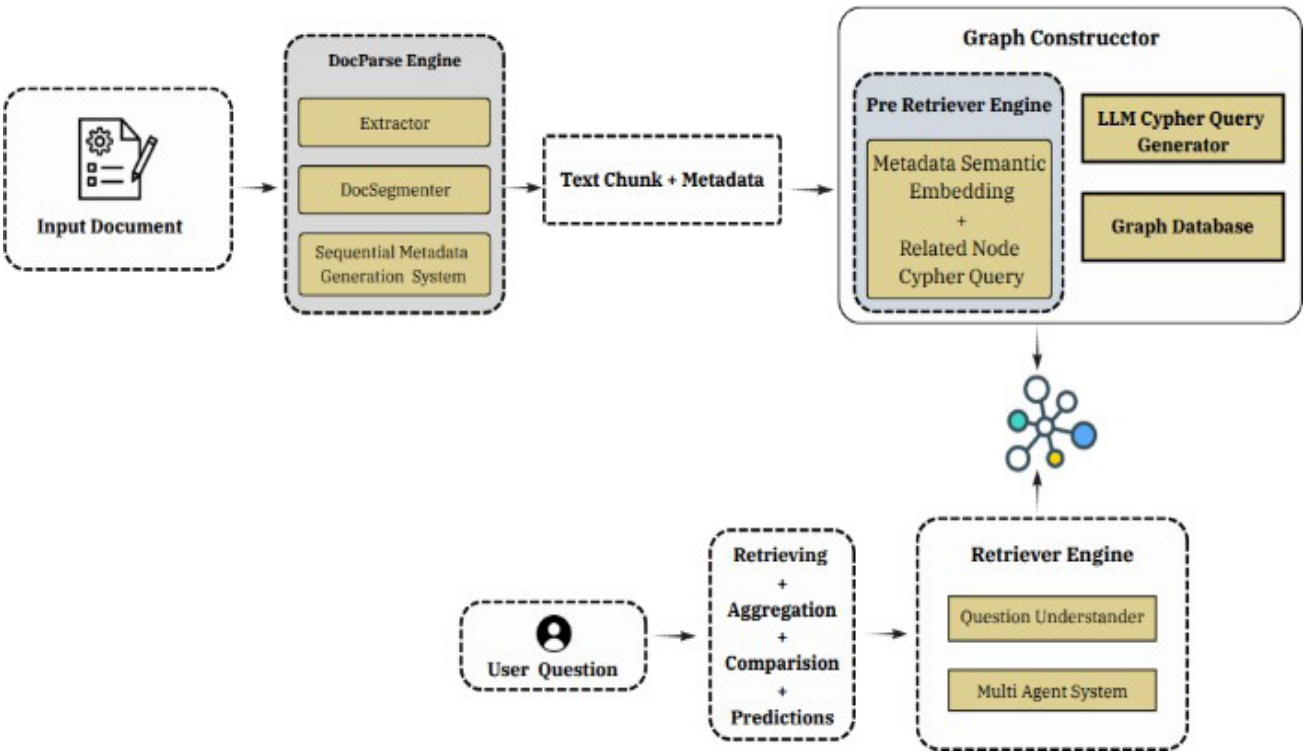

Fig. 1: Document to Knowledge Graph Question Answering System Architecture

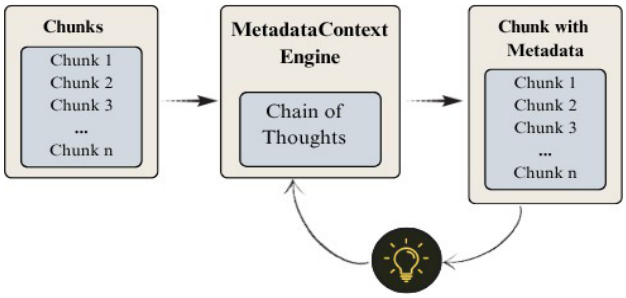

Fig. 2

C. Cross-Chunk Reference Linking

The system upholds a reference graph Gref (V,E) where:

- Each vi∈V is a chunk,
- Each eij∈E represents a semantic dependency (e.g., citations, figure references).

A hybrid reference linking methodology is used to identify reference links:

- Regex + Heuristics Patterns such as see Section 2.3, in Table 1.
- Named entity overlap of identifying shared entities.
- Dense embedding relatedness (e.g., SBERT) to detect paraphrased or unmarked dependencies

Each edge eij is given a relevance score and kept in case it is

larger than a threshold θ.

D. Metadata Schema and Output Format

The system is associated with a reference graph Gref=(V,E) wherein:

json

{

"chunk_id": "C_17",

"summary": "Experiments showed a 12% improvement post-optimization.",

"semantic_role": "Results",

"hierarchy_level": "Section 4 > Subsection 4.2", "refers_to": ["C_15", "F_3"],

"confidence": 0.92, "chunk_text": "..."

}

This metadata is stored in conjunction with the text thus permitting symbolic reasoning, graph building and trackable QA.

E. Context-Aware Processing

SMGS is concurrently in operation within a context window. It maintains the memory of the already processed chunks and changes dynamically the entity states and section hierarchy. This makes possible gradual improvement:

- Metadata in the past can be re-considered in case of contradictions or the development of dependencies.
- Final products are internationally uniform in massive documents.

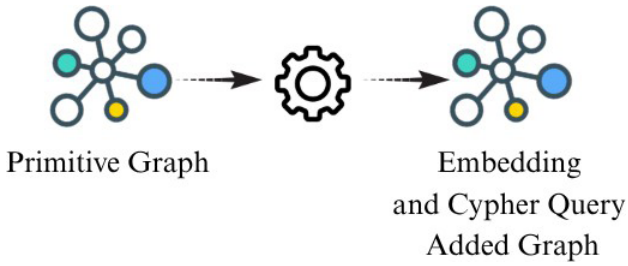

Fig. 3

A second Verifier Agent can be engaged in high stakes or regulated areas (e.g. legal, medical) to crossfields verify metadata fields and trigger flags of uncertainty.

Each output of the SMGS is a serial form and is sent to the Graph Constructor, the input of which are nodes in a hybrid symbolic-neural knowledge graph, each segment of which is modeled by a node. The nodes are linked by the references, shared objects and semantic roles, which enable the system to provide multi-hop querying and predictive reasoning.

1) Summary of DocParser Output

Upon completion of DocParser Engine, documents are transformed to collections of semantically structured enriched and token-aligned segments. Each of the segments is marked with the following information:

- Cloded section structure
- Summarized content
- Role classification
- References to figures, sections or entities.
- Confidence estimates

It is a structure that supports the construction of graphs thereafter with high support of interpretability, traceability as well as extensibility.

2) Graph Constructor

The Graph Constructor will transform the enriched text fragments and metadata to a structured knowledge graph. It is composed of two major elements:

- Pre-Retriever Engine
- LLM Cypher Query Generator

In the Pre-Retriever Engine every chunk would experience:

- *Metadata Semantic Embedding:* In each segment. The encoding of the content of each segment is concatenated with its metadata created by SMGS to obtain embeddings with transformer-based sentence encoders like Sentence-BERT. Such embeddings store and maintain both the textual and contextual relations.
- *Related Node Cypher Query Generation:* It is established how the prescribed ability is connected between a segment and another segment through the analysis of similarity of embedding and

alignment of metadata. Based on associations generated, Cypher queries are created to specify relationships in the graph, e.g. supports, explains, or refers to in the graph.

The processed nodes and the relationship between them are interred in a graph database, like Neo4j, and assume a hybrid structure where each node is an encapsulation of a text, metadata, and semantic embeddings. Cypher query generator is a language model (LLM)-based tool that takes queries expressed as natural language and transforms them into graph traversal instructions wherever search by methods of symbolic search can be used. The hybrid construction helps the symbolic reasoning with the aid of a graph traversal, and the dense semantic retrieval with the aid of embedding similarity, which lead to the more interpretable and accurate response to questions.

F. Retriever Engine and Multi-Agent Reasoning

The Retriever Engine is a retrieval that combines retrieval, comprehension, and reasoning to process queries by users. It comprises of the following important elements:

- Question Understander
- Multi-Agent System

Upon the reception of user query, the Question Understander component recognizes the intent of the query (e.g., factual, comparative or summarization based) and, where needed, decomposes complex queries into smaller sub-queries. Such polished queries are in turn sent to a Multi-Agent System of multiple specialised LLM agents:

- *Retriever Agent:* Intelligible selection of the nodes/subgraphs according to symbolic traversal as well as semantic similarity of the embedding.
- *Aggregator Agent:* Allows the combination of the multi-hop evidence into a coherent body of knowledge.
- *Comparison Agent:* Is used to handle contrastive and ranking oriented queries.
- *Prediction Agent:* Makes predictions or conclusions based on evidence retrieved backed up by the chain-of-thought (CoT) reasoning.

The engine generates responses that are given with traceable graph paths and extant text fragments, hence enhancing transparency and confidence in the results of the system. It is also modular and multi-agent architecture and thus scalable and domain adaptable, so task specific agents can be added where needed.

IV. Result & Experiment

A. Experimental Setup

The three different categories of documents used to evaluate the proposed Doc2GraphQA framework were:

- *Scientific Papers*: Structured PDF files containing citations, equations, and bibliographic references.
- *Business Reports (Annual & Financial):* Semi structured documents with tables, charts and descriptions.
- *Custom Domain-Specific Documents:* Unformatted textual information based on logistics reports and legal case documents.

The set of documents was converted into a knowledge graph each, which contains structured entities and relationships obtained with the help of the modular components of the framework. Neo4j 5.19, OpenAI GPT-4 APIs and Python 3.10 were used as the experimental setup. In order to test the robustness and performance, the system has been tested on four major metrics:

- *Execution Accuracy (EA) (%):* The degree of accuracy of the Cypher query.
- *Exact Match Accuracy (EMA) (%):* The accuracy of answers that have been returned.
- *Avg. Query Latency (ms):* Average system response time.
- *Adaptability Score (AS):* Has the ability to generalize between domains.

B. Quantitative Results

Table I shows the performance results of every module and its ablated counterparts. The entire framework was the most accurate in all metrics of evaluation. The experiments with ablation have shown that deleting the Query Generator or Retriver modules resulted in a significant decrease of the performance of the entire system, and therefore, they were of significant importance to it.

Table I: Quantitative Analysis of the Doc2GraphQA Framework

Variant / Ablation	EMA (%)	EA (%)	Avg. Query Latency (ms)	AS	Notes
Full Framework	91.8	94.2	110	0.94	All components enabled
No Query Generator	84.6	88.1	105	0.82	Uses static prompt templates
No Retriever	86.4	89.3	97	0.85	Misses global context retrieval
No Graph Ranker	88.7	91.4	93	0.89	Outputs broader subgraph
No Answer Verifier	89.0	91.6	94	0.90	Reduces post-query confidence

C. Dataset-Wise Performance

Table II provides data-specific findings that were used to test the flexibility of the framework to various content areas. The performance of the system remains constant with slight differences in query accuracy and response times, which confirms that the design is domain-agnostic.

Table II: Dataset-Specific Evaluation of Doc2GraphQA

Dataset	EMA (%)	EA (%)	Avg. Query Latency (ms)	AS
Scientific Papers	91.8	94.2	110	0.94
Business Reports	90.1	92.4	113	0.91
IEEE Project Abstracts	89.7	91.2	119	0.90

D. Graphical Representation

The relative performance of every ablation variant is shown in the figure below which is used to illustrate the contribution of every module to the overall performance of the entire framework.

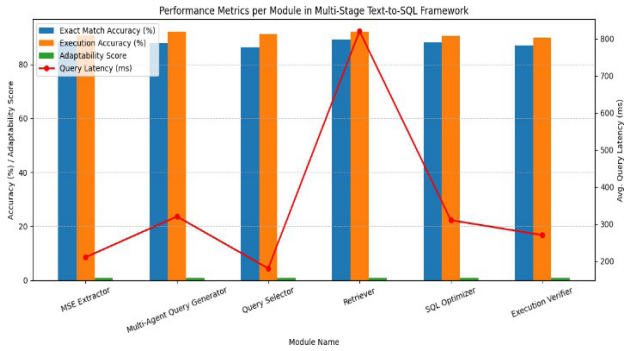

Fig. 4

V. Conclusion

The gig will be regarding Doc2GraphQA, a completely modular and scalable system that transforms raw documents into pretty graph-like things to be able to obtain clear and on-point answers. We have a DocSegmenter which understands Markdown, a SMGS with rich juicy metadata, a hybrid Graph Constructor, that can allow you to perform serious multi-hop reasoning over difficult documents. And we add a Retrizer Engine with a Multi-Agent System to make it able to multitask all types of questions and whether to be straight-forward or have a deeper understanding. The experiments indicate that Doc2GraphQA maintains a rather narrow context, connects related bits, and provides you with fully explainable and graph-grounded responses, which makes it is an effective approach towards long-document question and answer.

References

[1] Wang, Yu, Nedim Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. "Knowledge graph prompting for multi-document question answering." In *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 17, pp. 19206–19214. 2024.

[2] Hogan, Aidan, Xin Luna Dong, Denny Vrandečić, and Gerhard Weikum. "Large Language Models, Knowledge Graphs and SearchEngines: A Crossroads for Answering Users' Questions." *arXiv preprint arXiv:2501.06699* (2025).

[3] Yasunaga, Michihiro, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. "QA-GNN: Reasoning with language models and knowledge graphs for question answering." *arXiv preprint arXiv:2104.06378* (2021).

[4] Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler *et al.* "Retrieval-augmented generation for knowledge-intensive nlp tasks." *Advances in neural information processing systems* 33 (2020): 9459–9474.

[5] Kojima, Takeshi, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. "Large language models are zero-shot reasoners." *Advances in neural information processing systems* 35 (2022): 22199–22213.

[6] Ren, Hongyu, Hanjun Dai, Bo Dai, Xinyun Chen, Denny Zhou, Jure Leskovec, and Dale Schuurmans. "Smore: Knowledge graph completion and multi-hop reasoning in massive knowledge graphs." In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, pp. 1472–1482. 2022.

[7] Srivastava, Pragya, Manuj Malik, Vivek Gupta, Tanuja Ganu, and Dan Roth. "Evaluating LLMs' Mathematical Reasoning in Financial Document Question Answering." *arXiv preprint arXiv:2402.11194* (2024).

[8] Zhou, Zhanke, Rong Tao, Jianing Zhu, Yiwen Luo, Zengmao Wang, and Bo Han. "Can language models perform robust reasoning in chain-of-thought prompting with noisy rationales?." *Advances in Neural Information Processing Systems* 37 (2024): 123846–123910.

[9] Zhuang, Alex, Ge Zhang, Tianyu Zheng, Xinrun Du, Junjie Wang, Weiming Ren, Stephen W. Huang, Jie Fu, Xiang Yue, and Wenhu Chen. "Structlm: Towards building generalist models for structured knowledge grounding." *arXiv preprint arXiv:2402.16671* (2024).

[10] Khosravi, Asal, Zahed Rahmati, and Ali Vefghi. "Relational graph convolutional networks for sentiment analysis." *arXiv preprint arXiv:2404.13079* (2024).

[11] Fan, Wenqi, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. "A survey on rag meeting llms: Towards retrieval-augmented large language models." In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, pp. 6491-6501. 2024.

[12] Peng, Daowan, and Wei Wei. "Towards Deconfounded Visual Question Answering via Dual-causal Intervention." In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pp. 1867–1877. 2024.

[13] Efosa-Zuwa, Emmanuel, Olufunke Oladipupo, and Jelili Oyelade. "From Extraction to Reasoning: A Systematic Review of Algorithms in Multi-Document Summarization and QA." *Statistics, Optimization & Information Computing* 13, no. 6 (2025): 2529–2559.

[14] Rasal, Sumedh, and E. J. Hauer. "Navigating complexity: Orchestrated problem solving with multi-agent llms." *arXiv preprint arXiv:2402.16713* (2024).

[15] Mohamed, Ahmed Yasser, Mohamed Hisham Zain-elabdeen, Salwa Ahmed Sayed, Samaa Sabry Abdelwahab, Ziad Ezzat Mohamed, Zyad Mahmoud Ali, Yomna A. Kawashti, and Hanan Hindy. "SQLify Me: An Automated Text-To-SQL Query Generator." In *2024 34th International Conference on Computer Theory and Applications (ICCTA)*, pp. 297–302. IEEE, 2024.